2026-05-06-p2-f21-codex-approval-modes

P2-F21 — Codex Approval Modes Implementation Plan (FIRST Phase 2 feature)

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

> **Programme position:** F21 is the **first** Phase 2 feature of CLI-Agent Fusion. Task T01 advances PROGRESS.md from "Phase 1 complete" to "Phase 2 in flight" and creates `docs/improvements/07_phase_2_evidence.md` (the new evidence section for Phase 2).

Goal: Ship a real, end-to-end **4-mode codex-compatible approval system** for the HelixCode CLI agent. F21 adds an `internal/approval/` package with `ApprovalMode` (4 values: `suggest`, `auto-edit`, `full-auto`, `dangerously-bypass`) + `ApprovalLevel` (4 values: `read-only`, `edit`, `run`, `all`) + `Decision` value type + `ApprovalManager` (atomic-pointer mode + 4×4 matrix gate + F02 PolicyEngine + F14 SandboxManager + F19 Prompter integration) + `Selector` (flag > env > config > default `suggest`, mirrors F12 pattern). Extends `tools.Tool` interface with `RequiresApproval() approval.ApprovalLevel`; T05 migrates ~30 existing tools with explicit levels per spec §3.6 table. Wires through `tools/registry.go::Execute` as a NEW pre-execute branch (FIRST in chain, before F02). Adds a `/approval` slash command (`status` / `set <mode>` / `show <mode>`); NO cobra subcommand. Runtime mode change via `/approval set` is structurally enforced via `atomic.Pointer[ApprovalMode]` swap; the next `CheckApproval` call sees the new mode.

Architecture: New `internal/approval/` package with `types.go` (`ApprovalMode` + `ApprovalLevel` + `Action` + `Decision` + `ResolvedSource` + sentinel errors `ErrApprovalRequired`/`ErrUnknownMode`/`ErrSandboxRequired` + `ModeDescriptors` constant table for `/approval show`), `selector.go` (`SelectorInput` + `Select(input) (ApprovalMode, ResolvedSource, error)` + `ParseMode(s) (ApprovalMode, error)` — pure function; injected env closure), `manager.go` (`ManagerOptions` + `ApprovalManager` with `atomic.Pointer[ApprovalMode]` + `CheckApproval(ctx, tool, params) Decision` + `Set(mode)` + `Mode()` + `Source()` + `DefaultLevelEdit` embeddable struct + `YesNoPrompter` interface), `yaml_loader.go` (`LoadConfigFile(path)` mirroring F12's wizard-writer style). New `internal/commands/approval_command.go` for the `/approval` slash command. Two existing files get small additions: `internal/tools/registry.go` (1) `Tool` interface + `RequiresApproval() approval.ApprovalLevel` method, (2) `ToolRegistry` + `approvalMgr` field + `SetApprovalManager(mgr)`, (3) `Execute` gains a NEW pre-execute branch (FIRST in chain, before F02) calling `manager.CheckApproval`; `cmd/cli/main.go` (1) construct `approval.Select` + `approval.NewManager` adjacent to F02 + F14 + F19 wiring, (2) `--approval` pflag wiring, (3) `c.toolRegistry.SetApprovalManager(c.approvalMgr)` + register `/approval` slash. Plus ~30 one-line `RequiresApproval()` methods on existing tools per spec §3.6 table; tools that don't override embed `approval.DefaultLevelEdit` (LevelEdit safe-by-default).

Tech Stack: Go 1.26, testify v1.11, zap (already in `go.mod`), `gopkg.in/yaml.v3` (direct dep promoted in F20). **Zero new external deps** (atomic, errors, fmt, io/fs, os, path/filepath, strings, context, sync are all stdlib). `go mod tidy` after T07 must produce no diff in either `go.mod` or `go.sum`. T09's verification step asserts this loudly.

Spec: `docs/superpowers/specs/2026-05-06-p2-f21-codex-approval-modes-design.md` (commit `7128289`).

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F21 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/approval internal/commands/approval_command.go &&
        echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — F21 can degenerate in four ways: (a) the manager constructs cleanly + `/approval status` shows the mode + 100% unit-test coverage on `internal/approval/`, but `registry.Execute` never calls `CheckApproval`, so every tool runs regardless of mode (the gate is metadata, not enforcement); (b) `suggest` mode `CheckApproval` returns `Decision.Deny` correctly, but the registry ignores the deny value and proceeds to `tool.Execute`, so a `WriteFileTool` call mutates disk silently while the test only checks "did the test set up the manager" (the deny is computed-but-not-honoured); (c) `full-auto` + `LevelRun` sets `params["_helix_sandbox_required"] = true` but the registry's F14 dispatch ignores the marker and runs through plain `os/exec` (the sandbox-on claim is a UI promise without a runtime path); (d) `/approval set <mode>` mutates a struct field that `CheckApproval` doesn't re-read on entry (e.g., the mode is captured in a closure at `NewManager` time so the slash command's swap is invisible to subsequent calls). The four "what counts as approval system works" criteria — (1) real `WriteFileTool` invoked through real registry under `ModeSuggest` returns `ErrApprovalRequired` AND the target file does NOT exist on disk (PHASE-A byte evidence); (2) real shell tool under `ModeAutoEdit` with yes-replying Prompter runs (file/output observable); same tool with no-replying Prompter returns `ErrApprovalRequired` AND the exec wrapper's invocation count is 0 (PHASE-B); (3) real shell tool under `ModeFullAuto` routes through `sandbox.Manager.Execute` (sandbox spy invocation count == 1) AND the params marker `_helix_sandbox_network_allowed` is false (PHASE-C; skip-with-marker if bwrap unavailable); (4) before/after `manager.Set(ModeAutoEdit)` produces observably different `registry.Execute` outcomes for the same `WriteFileTool` call in the same goroutine (PHASE-D) — are each tested with both unit assertions AND a Challenge phase. PHASE-E additionally asserts F02 final-deny composition: `ModeDangerous` + an F02 rule denying `/etc/` writes still results in deny, with the file NOT created. The Challenge harness uses positive evidence: invocation counters (PHASE-A: WriteFileTool counter == 0 proves gate fired before tool; PHASE-B: exec counter == 0 in no-reply scenario), filesystem-state assertions (PHASE-A: `os.IsNotExist`; PHASE-B/D: `os.ReadFile` content equality), Prompter-replay equality (PHASE-B yes/no branches), sandbox-spy invocation count (PHASE-C count == 1), runtime-mode-change observable in tool outcome (PHASE-D), F02-final-deny composition (PHASE-E). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: the approval system is the safety surface every mutating tool flows through. F02 already gates per-rule (path globs, command patterns); F14 already sandboxes shell tool execution. F21 is the **mode posture** layer: a single user-chosen risk dial that gates whole

categories at once, so a user in `suggest` mode is structurally protected from any rogue mutation regardless of which specific tool the LLM picks. F21's discriminating tests are: (i) PHASE-A's `os.IsNotExist` assertion (proves real gate enforcement, not metadata-only deny); (ii) PHASE-B's exec-counter=0 in the no-reply branch (proves Prompter integration is real, not stubbed); (iii) PHASE-C's sandbox-spy count == 1 (proves the `full-auto` sandbox-required marker is actually honoured by F14's dispatch, not just stamped on the params); (iv) PHASE-D's before/after differential outcome (proves runtime mode change reaches `CheckApproval`'s entry path, not just a struct field); (v) PHASE-E's F02-final-deny under `ModeDangerous` (proves the composition rule § 4.4 holds — F21 is NOT a license to bypass F02). All five must produce positive evidence; none can be satisfied by absence-of-error.

Task list

- [ ] P2-F21-T01 — bootstrap Phase 2 evidence + advance PROGRESS to F21 (first Phase 2 feature)
- [ ] P2-F21-T02 — `internal/approval/types.go`: ApprovalMode + ApprovalLevel + Action + Decision + ResolvedSource + sentinels + ModeDescriptors table (TDD)
- [ ] P2-F21-T03 — `internal/approval/selector.go`: flag/env/config precedence + ParseMode (TDD; mirror F12 Selector)
- [ ] P2-F21-T04 — `internal/approval/manager.go`: ApprovalManager with atomic-pointer mode + 4×4 matrix gate + F02/F14/F19 integration + DefaultLevelEdit + YAML loader (TDD)
- [ ] P2-F21-T05 — Extend `tools.Tool` interface with `RequiresApproval()` `approval.ApprovalLevel` + apply to all ~30 existing tools (TDD; coverage table-test)
- [ ] P2-F21-T06 — `internal/commands/approval_command.go`: /approval slash (status/set/show) (TDD)
- [ ] P2-F21-T07 — main.go wiring: ApprovalManager construction + `--approval` pflag + registry hook + /approval registration (TDD with integration test)
- [ ] P2-F21-T08 — Challenge harness: 5 phases (one per mode + runtime change + F02 composition) with positive byte evidence
- [ ] P2-F21-T09 — F21 close-out + push 4 remotes non-force

Task 1: Bootstrap Phase 2 evidence

Create `docs/improvements/07_phase_2_evidence.md` (mirroring `06_phase_1_evidence.md` shape; first entry will be F21's evidence in T08). Append F21 evidence section header (spec `7128289`). Update PROGRESS.md current focus from "Phase 1 of CLI-Agent Fusion programme COMPLETE" to "Phase 2 of CLI-Agent Fusion programme:

F21 (Codex Approval Modes) in flight". Insert F21 task list (9 items). Verify zero new external deps — `gopkg.in/yaml.v3` is already direct (F20 promotion).

```
cd HelixCode && grep "yaml.v3" go.mod  # confirm direct (post-
        F20)
cd HelixCode && grep -E "approval|RequiresApproval" go.mod &&
        echo "UNEXPECTED" || echo "clean"
```

Commit: `docs(P2-F21-T01): bootstrap Phase 2 evidence + advance PROGRESS to F21 (Codex Approval Modes).`

Task 2: types.go (TDD)

Files: new `helix_code/internal/approval/types.go`, new `helix_code/internal/approval/types_test.go`.

Define: - `ApprovalMode int` enum + 4 values (`ModeSuggest` / `ModeAutoEdit` / `ModeFullAuto` / `ModeDangerous`) + private `numModes` sentinel. - `ApprovalMode.String()` returning codex-compatible names ("suggest", "auto-edit", "full-auto", "dangerously-bypass"). - `ParseMode(s string) (ApprovalMode, error)` — accepts kebab-case + underscore variants; returns `ErrUnknownMode` for unknown. - `ApprovalLevel int` enum + 4 values (`LevelReadOnly` / `LevelEdit` / `LevelRun` / `LevelAll`) + private `numLevels` sentinel + `String()` returning "read-only"/"edit"/"run"/"all". - `Action int` enum + 3 values (`ActionAllow` / `ActionDeny` / `ActionPrompt`) + `String()` returning "allow"/"deny"/"prompt". - `Decision struct { Action Action; Reason string; SandboxRequired bool }`.- `ResolvedSource string` type + 4 constants (`SourceFlag` / `SourceEnv` / `SourceConfig` / `SourceDefault`). - `ModeDescriptor struct { Mode ApprovalMode; Summary string; Description string; Matrix [numLevels]Action }`. - `ModeDescriptors [numModes]ModeDescriptor` constant table per spec §3.4 (4×4 matrix verbatim). - Error sentinels (`ErrApprovalRequired`, `ErrUnknownMode`, `ErrSandboxRequired`).

Failing tests FIRST:

```
func TestApprovalMode_String_AllFour(t *testing.T) {
    require.Equal(t, "suggest",            ModeSuggest.String())
    require.Equal(t, "auto-edit",          ModeAutoEdit.String())
    require.Equal(t, "full-auto",          ModeFullAuto.String())
    require.Equal(t, "dangerously-bypass",
        ModeDangerous.String())
}

func TestApprovalMode_ParseMode_OK_KebabAndUnderscore(t
        *testing.T) {
    cases := map[string]ApprovalMode{
        "suggest": ModeSuggest, "auto-edit": ModeAutoEdit,
        "auto_edit": ModeAutoEdit,
        "full-auto": ModeFullAuto, "full_auto": ModeFullAuto,
```

```
        "dangerously-bypass": ModeDangerous,
        "dangerously_bypass": ModeDangerous,
    }
    for s, want := range cases {
        got, err := ParseMode(s)
        require.NoError(t, err, s); require.Equal(t, want, got,
        s)
    }
}

func TestApprovalMode_ParseMode_Unknown_Err(t *testing.T) {
    _, err := ParseMode("bogus")
    require.ErrorIs(t, err, ErrUnknownMode)
}

func TestApprovalLevel_String_AllFour(t *testing.T) {
    require.Equal(t, "read-only", LevelReadOnly.String())
    require.Equal(t, "edit",      LevelEdit.String())
    require.Equal(t, "run",       LevelRun.String())
    require.Equal(t, "all",       LevelAll.String())
}

func TestAction_String_AllThree(t *testing.T) {
    require.Equal(t, "allow",  ActionAllow.String())
    require.Equal(t, "deny",   ActionDeny.String())
    require.Equal(t, "prompt", ActionPrompt.String())
}

func TestModeDescriptors_TableShape_4x4Matrix(t *testing.T) {
    // Pin §3.4's 4×4 matrix byte-for-byte.
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeSuggest].Matrix[LevelReadOnly])
    require.Equal(t, ActionDeny,
        ModeDescriptors[ModeSuggest].Matrix[LevelEdit])
    require.Equal(t, ActionDeny,
        ModeDescriptors[ModeSuggest].Matrix[LevelRun])
    require.Equal(t, ActionDeny,
        ModeDescriptors[ModeSuggest].Matrix[LevelAll])

    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeAutoEdit].Matrix[LevelReadOnly])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeAutoEdit].Matrix[LevelEdit])
    require.Equal(t, ActionPrompt,
        ModeDescriptors[ModeAutoEdit].Matrix[LevelRun])
    require.Equal(t, ActionDeny,
        ModeDescriptors[ModeAutoEdit].Matrix[LevelAll])
```

```
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeFullAuto].Matrix[LevelReadOnly])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeFullAuto].Matrix[LevelEdit])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeFullAuto].Matrix[LevelRun]) //
        sandbox forced separately
    require.Equal(t, ActionDeny,
        ModeDescriptors[ModeFullAuto].Matrix[LevelAll])

    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeDangerous].Matrix[LevelReadOnly])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeDangerous].Matrix[LevelEdit])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeDangerous].Matrix[LevelRun])
    require.Equal(t, ActionAllow,
        ModeDescriptors[ModeDangerous].Matrix[LevelAll])
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{ErrApprovalRequired,
        ErrUnknownMode, ErrSandboxRequired} {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}
```

Subject: `feat(P2-F21-T02): approval types - Mode + Level + Action + Decision + ModeDescriptors table.`

Task 3: selector.go (TDD; mirror F12 Selector)

Files: new `helix_code/internal/approval/selector.go`, new `helix_code/internal/approval/selector_test.go`.

`selector.go`:

```
type SelectorInput struct {
    Flag       string              // --approval=<value>; ""
        means flag not set
    Env        func(string)
        string // env lookup closure (default os.Getenv)
    ConfigPath string              // ~/.config/helixcode/
        approval.yaml; "" means no config
    Filesystem
        fs.FS              // injected for tests; default
        os.DirFS("/")
```

```
}

func Select(input SelectorInput) (ApprovalMode, ResolvedSource,
        error) {
    // 1. Flag (highest precedence)
    if input.Flag != "" {
        m, err := ParseMode(input.Flag)
        if err != nil {
            // log warning + fall through
            return selectFromEnvOrConfig(input)
        }
        return m, SourceFlag, nil
    }
    return selectFromEnvOrConfig(input)
}

func selectFromEnvOrConfig(input SelectorInput) (ApprovalMode,
        ResolvedSource, error) {
    // 2. Env var
    if input.Env != nil {
        if v := input.Env("HELIXCODE_APPROVAL"); v != "" {
            m, err := ParseMode(v)
            if err == nil {
                return m, SourceEnv, nil
            }
        }
    }
    // 3. Config file
    if input.ConfigPath != "" && input.Filesystem != nil {
        if m, ok := loadModeFromYAML(input.Filesystem,
        input.ConfigPath); ok {
            return m, SourceConfig, nil
        }
    }
    // 4. Default
    return ModeSuggest, SourceDefault, nil
}
```

Failing tests FIRST (table-driven; mirror F12 selector pattern):

```
func fakeEnv(m map[string]string) func(string) string {
    return func(k string) string { return m[k] }
}

func TestSelector_FlagWins_OverEnvAndConfig(t *testing.T) {
    m, src, err := Select(SelectorInput{
        Flag: "auto-edit",
        Env:  fakeEnv(map[string]string{"HELIXCODE_APPROVAL":
        "full-auto"}),
```

```
        // ConfigPath set but flag wins
    })
    require.NoError(t, err)
    require.Equal(t, ModeAutoEdit, m)
    require.Equal(t, SourceFlag, src)
}

func TestSelector_EnvWins_WhenFlagEmpty(t *testing.T) {
    m, src, err := Select(SelectorInput{
        Env: fakeEnv(map[string]string{"HELIXCODE_APPROVAL":
        "full-auto"}),
    })
    require.NoError(t, err)
    require.Equal(t, ModeFullAuto, m)
    require.Equal(t, SourceEnv, src)
}

func TestSelector_ConfigWins_WhenFlagAndEnvEmpty(t *testing.T) {
    fsys := fstest.MapFS{
        "approval.yaml": &fstest.MapFile{Data: []byte("mode:
        dangerously-bypass\n")},
    }
    m, src, err := Select(SelectorInput{
        Env:        fakeEnv(map[string]string{}),
        ConfigPath: "approval.yaml",
        Filesystem: fsys,
    })
    require.NoError(t, err)
    require.Equal(t, ModeDangerous, m)
    require.Equal(t, SourceConfig, src)
}

func TestSelector_DefaultSuggest_WhenAllEmpty(t *testing.T) {
    m, src, err := Select(SelectorInput{Env:
        fakeEnv(map[string]string{})})
    require.NoError(t, err)
    require.Equal(t, ModeSuggest, m)
    require.Equal(t, SourceDefault, src)
}

func TestSelector_UnknownFlag_FallsThroughToEnv(t *testing.T) {
    m, src, err := Select(SelectorInput{
        Flag: "garbage",
        Env:  fakeEnv(map[string]string{"HELIXCODE_APPROVAL":
        "auto-edit"}),
    })
    require.NoError(t, err)
    require.Equal(t, ModeAutoEdit, m)
```

```
    require.Equal(t, SourceEnv, src)
}

func TestSelector_UnknownEnv_FallsThroughToDefault(t *testing.T)
        {
    m, src, err := Select(SelectorInput{
        Env: fakeEnv(map[string]string{"HELIXCODE_APPROVAL":
        "garbage"}),
    })
    require.NoError(t, err)
    require.Equal(t, ModeSuggest, m)
    require.Equal(t, SourceDefault, src)
}

func TestSelector_UnknownYAMLMode_FallsThroughToDefault(t
        *testing.T) {
    fsys := fstest.MapFS{
        "approval.yaml": &fstest.MapFile{Data: []byte("mode:
        bogus\n")},
    }
    m, src, _ := Select(SelectorInput{
        Env:        fakeEnv(map[string]string{}),
        ConfigPath: "approval.yaml",
        Filesystem: fsys,
    })
    require.Equal(t, ModeSuggest, m)
    require.Equal(t, SourceDefault, src)
}
```

Subject: `feat(P2-F21-T03): approval selector - flag/env/config/default precedence (mirror F12).`

Task 4: manager.go + yaml_loader.go (TDD; F02/F14/F19 integration)

Files: new `helix_code/internal/approval/manager.go`, new `helix_code/internal/approval/manager_test.go`, new `helix_code/internal/approval/yaml_loader.go`, new `helix_code/internal/approval/yaml_loader_test.go`.

`manager.go`:

```
type ManagerOptions struct {
    Mode         ApprovalMode
    Source       ResolvedSource
    PolicyEngine *confirmation.PolicyEngine // F02; nil OK for
        tests
```

```
    SandboxMgr   *sandbox.SandboxManager    // F14; nil OK for
        tests (CheckApproval still works)
    Prompter     YesNoPrompter              // F19 wrapper;
        required for ActionPrompt branch
    Logger       *zap.Logger                // optional; nil →
        no-op
    SleepFunc    func(time.Duration)        // optional; default
        time.Sleep — for dangerous-pause seam
}

type YesNoPrompter interface {
    PromptYesNo(ctx context.Context, question string, defaultYes
        bool) (bool, error)
}

type Tool interface {
    Name() string
    RequiresApproval() ApprovalLevel
}

type ApprovalManager struct {
    mode      atomic.Pointer[ApprovalMode]
    source    ResolvedSource
    pe        *confirmation.PolicyEngine
    sm        *sandbox.SandboxManager
    prompter  YesNoPrompter
    log       *zap.Logger
    sleepFunc func(time.Duration)
}

func NewManager(opts ManagerOptions) *ApprovalManager {
    m := &ApprovalManager{
        source: opts.Source, pe: opts.PolicyEngine, sm:
        opts.SandboxMgr,
        prompter: opts.Prompter, log: opts.Logger,
        sleepFunc: opts.SleepFunc,
    }
    if m.sleepFunc == nil { m.sleepFunc = time.Sleep }
    mode := opts.Mode
    m.mode.Store(&mode)
    return m
}

func (m *ApprovalManager) CheckApproval(ctx
        context.Context, tool Tool, params
        map[string]interface{}) Decision {
    mode := *m.mode.Load()
    level := tool.RequiresApproval()
```

```
    if mode < 0 || mode >= numModes || level < 0 || level >=
        numLevels {
        return Decision{Action: ActionDeny, Reason: "approval:
        invalid mode or level"}
    }
    action := ModeDescriptors[mode].Matrix[level]
    switch action {
    case ActionAllow:
        sandbox := mode == ModeFullAuto && level == LevelRun
        return Decision{Action: ActionAllow, SandboxRequired:
        sandbox}
    case ActionDeny:
        return Decision{Action: ActionDeny, Reason: fmt.Sprintf(
            "approval required: %s mode forbids %s-level
        operations; switch to %s or higher",
            mode, level, minimumModeForLevel(level))}
    case ActionPrompt:
        return Decision{Action: ActionPrompt, Reason:
        fmt.Sprintf(
            "%s mode requires confirmation for %s-level
        operations", mode, level)}
    }
    return Decision{Action: ActionDeny, Reason: "approval:
        unreachable"}
}

func (m *ApprovalManager) Mode() ApprovalMode    { return
        *m.mode.Load() }
func (m *ApprovalManager) Source() ResolvedSource { return
        m.source }

func (m *ApprovalManager) Set(mode ApprovalMode) {
    if mode == ModeDangerous {
        if m.log != nil {
            m.log.Warn("approval mode changed to dangerously-
        bypass; ALL approval checks disabled")
        }
        m.sleepFunc(2 * time.Second)
    }
    m.mode.Store(&mode)
}

type DefaultLevelEdit struct{}
func (DefaultLevelEdit) RequiresApproval() ApprovalLevel {
        return LevelEdit }

func minimumModeForLevel(l ApprovalLevel) ApprovalMode {
    switch l {
```

```
    case LevelReadOnly: return ModeSuggest
    case LevelEdit:     return ModeAutoEdit
    case LevelRun:      return
        ModeAutoEdit  // auto-edit prompts; full-auto allows
    case LevelAll:      return ModeDangerous
    }
    return ModeDangerous
}
```

yaml_loader.go:

```
type approvalConfig struct {
    Mode string `yaml:"mode"`
}

func LoadConfigFile(path string) (ApprovalMode, error) { /* uses
        os.ReadFile + yaml.Unmarshal */ }

func loadModeFromYAML(fsys fs.FS, path string) (ApprovalMode,
        bool) {
    data, err := fs.ReadFile(fsys, path)
    if err != nil { return 0, false }
    var c approvalConfig
    if err := yaml.Unmarshal(data, &c); err != nil { return 0,
        false }
    if c.Mode == "" { return 0, false }
    m, err := ParseMode(c.Mode)
    if err != nil { return 0, false }
    return m, true
}
```

Failing tests FIRST (16 entries for the 4×4 matrix; runtime swap; F02 nil-safe; sandbox-required marker; etc.):

```
type fakeTool struct {
    name  string
    level ApprovalLevel
    calls int
}
func (f *fakeTool) Name() string                       { return
        f.name }
func (f *fakeTool) RequiresApproval() ApprovalLevel    { return
        f.level }

func TestManager_CheckApproval_4x4Matrix_ExhaustiveTable(t
        *testing.T) {
    cases := []struct {
        mode    ApprovalMode
        level   ApprovalLevel
```

```
        wantAct Action
    }{
        {ModeSuggest, LevelReadOnly, ActionAllow},
        {ModeSuggest, LevelEdit,     ActionDeny},
        {ModeSuggest, LevelRun,      ActionDeny},
        {ModeSuggest, LevelAll,      ActionDeny},
        {ModeAutoEdit, LevelReadOnly, ActionAllow},
        {ModeAutoEdit, LevelEdit,     ActionAllow},
        {ModeAutoEdit, LevelRun,      ActionPrompt},
        {ModeAutoEdit, LevelAll,      ActionDeny},
        {ModeFullAuto, LevelReadOnly, ActionAllow},
        {ModeFullAuto, LevelEdit,     ActionAllow},
        {ModeFullAuto, LevelRun,      ActionAllow}, //
        SandboxRequired=true
        {ModeFullAuto, LevelAll,      ActionDeny},
        {ModeDangerous, LevelReadOnly, ActionAllow},
        {ModeDangerous, LevelEdit,     ActionAllow},
        {ModeDangerous, LevelRun,      ActionAllow},
        {ModeDangerous, LevelAll,      ActionAllow},
    }
    for _, tc := range cases {
        m := NewManager(ManagerOptions{Mode: tc.mode})
        dec := m.CheckApproval(context.Background(),
            &fakeTool{name: "t", level: tc.level}, nil)
        require.Equal(t, tc.wantAct, dec.Action,
            "mode=%v level=%v", tc.mode, tc.level)
    }
}

func TestManager_CheckApproval_FullAutoRun_SandboxRequiredTrue(t
        *testing.T) {
    m := NewManager(ManagerOptions{Mode: ModeFullAuto})
    dec := m.CheckApproval(context.Background(),
        &fakeTool{name: "shell", level: LevelRun}, nil)
    require.Equal(t, ActionAllow, dec.Action)
    require.True(t, dec.SandboxRequired)
}

func TestManager_CheckApproval_AutoEditEdit_SandboxRequiredFalse(t
        *testing.T) {
    m := NewManager(ManagerOptions{Mode: ModeAutoEdit})
    dec := m.CheckApproval(context.Background(),
        &fakeTool{name: "write", level: LevelEdit}, nil)
    require.Equal(t, ActionAllow, dec.Action)
    require.False(t, dec.SandboxRequired)
}
```

```
func TestManager_CheckApproval_SuggestEdit_DenyReasonHasMinMode(t
        *testing.T) {
    m := NewManager(ManagerOptions{Mode: ModeSuggest})
    dec := m.CheckApproval(context.Background(),
        &fakeTool{name: "write", level: LevelEdit}, nil)
    require.Equal(t, ActionDeny, dec.Action)
    require.Contains(t, dec.Reason, "approval required")
    require.Contains(t, dec.Reason, "auto-edit")
}

func TestManager_Set_AtomicSwap_NextCallSeesNewMode(t
        *testing.T) {
    m := NewManager(ManagerOptions{Mode: ModeSuggest})
    tool := &fakeTool{name: "write", level: LevelEdit}

    dec1 := m.CheckApproval(context.Background(), tool, nil)
    require.Equal(t, ActionDeny, dec1.Action)

    m.Set(ModeAutoEdit)

    dec2 := m.CheckApproval(context.Background(), tool, nil)
    require.Equal(t, ActionAllow, dec2.Action)
}

func TestManager_Set_DangerousBypass_PauseSeamFires(t
        *testing.T) {
    var slept time.Duration
    m := NewManager(ManagerOptions{
        Mode:      ModeSuggest,
        SleepFunc: func(d time.Duration) { slept = d },
    })
    m.Set(ModeDangerous)
    require.Equal(t, 2*time.Second, slept)
    require.Equal(t, ModeDangerous, m.Mode())
}

func TestManager_NilPolicyEngine_OK(t *testing.T) {
    m := NewManager(ManagerOptions{Mode: ModeAutoEdit,
        PolicyEngine: nil})
    dec := m.CheckApproval(context.Background(),
        &fakeTool{name: "t", level: LevelEdit}, nil)
    require.Equal(t, ActionAllow, dec.Action)
}

func TestDefaultLevelEdit_RequiresApproval_LevelEdit(t
        *testing.T) {
    var d DefaultLevelEdit
    require.Equal(t, LevelEdit, d.RequiresApproval())
```

```
}

// YAML loader tests
func TestLoadConfigFile_OK_AllFourModes(t *testing.T) {
    cases := map[string]ApprovalMode{
        "suggest": ModeSuggest, "auto-edit": ModeAutoEdit,
        "full-auto": ModeFullAuto, "dangerously-bypass":
        ModeDangerous,
    }
    for s, want := range cases {
        f := writeTempYAML(t, "mode: "+s+"\n")
        m, err := LoadConfigFile(f)
        require.NoError(t, err); require.Equal(t, want, m)
    }
}
```

Subject: `feat(P2-F21-T04): approval manager + yaml loader - 4x4 matrix gate + atomic swap + nil-safe (TDD).`

Task 5: Tool.RequiresApproval interface extension + migrate ~30 existing tools (TDD)

Files: modify `helix_code/internal/tools/registry.go` (add `RequiresApproval() approval.ApprovalLevel` to `Tool` interface); modify ~30 existing tool files to add a one-line `RequiresApproval()` method per spec §3.6 table; new `helix_code/internal/tools/registry_requires_approval_test.go` (table-driven coverage test).

Steps:

1. Add `RequiresApproval() approval.ApprovalLevel` method to the `Tool` interface in `registry.go`.
2. Walk every tool registered in `buildToolList` (or its equivalent) and add a one-line method per spec §3.6 table:
    - **Read-only tools**: `func (t *Foo) RequiresApproval() approval.ApprovalLevel { return approval.LevelReadOnly }` for `read_file`, `list_directory`, `glob`, `grep`, `find`, `repomap`, `mapping_query`, `lsp_query`, `browser_read` variants, `ask_user`, `theme_*`.
    - **Edit-level tools** (default): tools that don't override embed `approval.DefaultLevelEdit` for safe-by-default. For `write_file`, `multi_edit`, `smart_edit`, `notebook_edit`, `mapping_edit`, `plan_mode_edit`, `git_*`, `mcp_*` (default), explicit override or embedding.
    - **Run-level tools**: `LevelRun` for `shell`, `shell_sandboxed`, `interactive_command`, `browser_navigate/click/type/evaluate`, `browser_run_code_unsafe`.
    - **All-level tools**: `LevelAll` for `subagent_dispatch` (F15), `permission_rule_write` (F02).

Failing test FIRST (table-driven coverage):

```
func TestToolRegistryRequiresApprovalCoverage(t *testing.T) {
    // Register every tool in production via the standard
        buildToolList path.
    reg := NewToolRegistry()
    if err := buildToolList(reg, /* ... real deps with nil-safe
        defaults ... */); err != nil {
        t.Fatalf("buildToolList: %v", err)
    }

    expected := map[string]approval.ApprovalLevel{
        "read_file":           approval.LevelReadOnly,
        "list_directory":      approval.LevelReadOnly,
        "glob":                approval.LevelReadOnly,
        "grep":                approval.LevelReadOnly,
        "repomap":             approval.LevelReadOnly,
        "lsp_query":           approval.LevelReadOnly,
        "browser_snapshot":    approval.LevelReadOnly,
        "ask_user":            approval.LevelReadOnly,

        "write_file":          approval.LevelEdit,
        "multi_edit":          approval.LevelEdit,
        "smart_edit":          approval.LevelEdit,
        "notebook_edit":       approval.LevelEdit,

        "shell":               approval.LevelRun,
        "shell_sandboxed":     approval.LevelRun,
        "browser_navigate":    approval.LevelRun,
        "browser_click":       approval.LevelRun,

        "subagent_dispatch": approval.LevelAll,
    }

    for name, want := range expected {
        tool, ok := reg.Get(name)
        if !ok { t.Errorf("tool %q not registered", name);
        continue }
        got := tool.RequiresApproval()
        require.Equal(t, want, got, "tool=%s", name)
    }

    // Bonus: every registered tool MUST return a level in the 4-
        value enum.
    for _, name := range reg.ListNames() {
        tool, _ := reg.Get(name)
        l := tool.RequiresApproval()
        require.GreaterOrEqual(t, int(l), 0)
        require.Less(t, int(l), int(approval.NumLevels()))  //
        exposed via package helper
```

```
    }
}
```

Non-obvious call: tools that don't appear in spec §3.6's explicit table embed `approval.DefaultLevelEdit` (which provides `RequiresApproval() ApprovalLevel { return LevelEdit }`). This is the safe-by-default rule (§11 #3 of the spec). The coverage test enumerates the expected-level table; any tool registered in `buildToolList` that doesn't appear in the table is required to return `LevelEdit` (the default), and the test asserts this. Future tool additions that forget to override the level get `LevelEdit` automatically — gated, not bypassed.

Subject: `feat(P2-F21-T05): Tool.RequiresApproval interface + migrate ~30 existing tools (safe-default LevelEdit).`

Task 6: /approval slash command (TDD)

Files: new `helix_code/internal/commands/approval_command.go`, new `helix_code/internal/commands/approval_command_test.go`.

`approval_command.go`:

```
type ApprovalCommand struct {
    mgr *approval.ApprovalManager
}

func NewApprovalCommand(mgr *approval.ApprovalManager)
        *ApprovalCommand {
    return &ApprovalCommand{mgr: mgr}
}

func (c *ApprovalCommand) Name() string         { return
        "approval" }
func (c *ApprovalCommand) Aliases() []string    { return nil }
func (c *ApprovalCommand) Description() string {
    return "Show or change the active approval mode (suggest/
        auto-edit/full-auto/dangerously-bypass)."
}
func (c *ApprovalCommand) Usage() string { return "/approval
        [status|set <mode>|show <mode>]" }

func (c *ApprovalCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error) {
    args := cc.Args
    sub := "status"
    if len(args) > 0 { sub = args[0] }
    switch sub {
    case "status":
        return c.status(), nil
    case "set":
```

```
        if len(args) < 2 { return nil, fmt.Errorf("/approval set
        <mode>") }
        return c.set(args[1])
    case "show":
        if len(args) < 2 { return nil,
        fmt.Errorf("/approval show <mode>") }
        return c.show(args[1])
    default:
        return nil, fmt.Errorf("/approval: unknown subcommand %q
        (want status|set|show)", sub)
    }
}

func (c *ApprovalCommand) status() *CommandResult { /* mode +
        source + per-level matrix */ }
func (c *ApprovalCommand) set(modeStr string) (*CommandResult,
        error) {
    m, err := approval.ParseMode(modeStr)
    if err != nil { return nil, err }
    if m == approval.ModeDangerous {
        // print warning line BEFORE Set (which sleeps 2s)
        // ...
    }
    c.mgr.Set(m)
    return &CommandResult{Output: fmt.Sprintf("approval mode →
        %s", m)}, nil
}
func (c *ApprovalCommand) show(modeStr string) (*CommandResult,
        error) {
    m, err := approval.ParseMode(modeStr)
    if err != nil { return nil, err }
    desc := approval.ModeDescriptors[m]
    // render desc.Description + desc.Matrix
    // ...
}
```

Failing tests FIRST:

```
func TestApprovalCommand_Name_IsApproval(t *testing.T) {
    require.Equal(t, "approval", NewApprovalCommand(nil).Name())
}

func TestApprovalCommand_Status_PrintsActiveModeAndSource(t
        *testing.T) {
    mgr := approval.NewManager(approval.ManagerOptions{
        Mode:   approval.ModeAutoEdit,
        Source: approval.SourceFlag,
    })
    cmd := NewApprovalCommand(mgr)
```

```
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "auto-edit")
    require.Contains(t, res.Output, "flag")
}

func TestApprovalCommand_Set_OK_ChangesMode(t *testing.T) {
    mgr := approval.NewManager(approval.ManagerOptions{Mode:
        approval.ModeSuggest})
    cmd := NewApprovalCommand(mgr)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"set", "auto-edit"}})
    require.NoError(t, err)
    require.Equal(t, approval.ModeAutoEdit, mgr.Mode())
}

func TestApprovalCommand_Set_UnknownMode_Err(t *testing.T) {
    mgr := approval.NewManager(approval.ManagerOptions{Mode:
        approval.ModeSuggest})
    cmd := NewApprovalCommand(mgr)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"set", "garbage"}})
    require.ErrorIs(t, err, approval.ErrUnknownMode)
}

func TestApprovalCommand_Set_Dangerous_PrintsWarning(t
        *testing.T) {
    var slept time.Duration
    mgr := approval.NewManager(approval.ManagerOptions{
        Mode: approval.ModeSuggest,
        SleepFunc: func(d time.Duration) { slept = d },
    })
    cmd := NewApprovalCommand(mgr)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"set", "dangerously-
        bypass"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "ALL approval checks")
    require.Equal(t, 2*time.Second, slept)
}

func TestApprovalCommand_Show_AllFourModes_RendersDescriptor(t
        *testing.T) {
    cmd :=
        NewApprovalCommand(approval.NewManager(approval.ManagerOptions{}))
    for _, m := range []string{"suggest", "auto-edit", "full-
        auto", "dangerously-bypass"} {
```

```
        res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"show", m}})
        require.NoError(t, err)
        require.Contains(t, res.Output, m)
    }
}

func TestApprovalCommand_Show_UnknownMode_Err(t *testing.T) {
    cmd :=
        NewApprovalCommand(approval.NewManager(approval.ManagerOptions{}))
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"show", "nope"}})
    require.ErrorIs(t, err, approval.ErrUnknownMode)
}
```

Subject: `feat(P2-F21-T06): /approval slash command (status/set/show) with dangerous-pause warning.`

Task 7: main.go wiring + integration test (TDD)

Files: modify `helix_code/cmd/cli/main.go` (wiring); new `helix_code/tests/integration/approval_test.go` (`//go:build integration`).

`main.go` changes (additive only; no new selectors needed beyond approval.Selector — F12's pattern is reused for shape):

1. Add `--approval=<mode>` to existing pflag set (one new line near other flag definitions).
2. Construct selector + manager AFTER F02 + F14 + F19 wiring is complete:

```
approvalMode, approvalSource, _ :=
        approval.Select(approval.SelectorInput{
    Flag:       *flagApproval,
    Env:        os.Getenv,
    ConfigPath: filepath.Join(xdgConfigHome(), "helixcode/
        approval.yaml"),
    Filesystem: os.DirFS("/"),
})

// Force-fail-fast: full-auto requires sandbox.
if approvalMode == approval.ModeFullAuto && !
        c.sandboxMgr.Capabilities().HasSandbox {
    return fmt.Errorf("%w: full-auto requires bubblewrap or
        native sandbox; "+

        "switch with --approval or HELIXCODE_APPROVAL, or install
        bwrap",
        approval.ErrSandboxRequired)
}
```

```
c.approvalMgr = approval.NewManager(approval.ManagerOptions{
    Mode:         approvalMode,
    Source:       approvalSource,
    PolicyEngine: c.policyEngine, // F02
    SandboxMgr:   c.sandboxMgr,   // F14
    Prompter:     yesnoAdapter(c.prompter), // wrapper around
        F19's Prompter (5-line adapter)
    Logger:       c.logger,
})

c.toolRegistry.SetApprovalManager(c.approvalMgr)

if regErr :=
        c.commandRegistry.Register(commands.NewApprovalCommand(c.approvalM
        regErr != nil {
    log.Printf("approval: register slash command failed: %v",
        regErr)
}
```

1. Modify `internal/tools/registry.go::Execute` to add the F21 gate as the FIRST pre-execute branch (before F02):

```
// F21 gate (NEW; FIRST in chain)
if r.approvalMgr != nil {
    dec := r.approvalMgr.CheckApproval(ctx, tool, params)
    switch dec.Action {
    case approval.ActionDeny:
        return nil, fmt.Errorf("%w: %s",
        approval.ErrApprovalRequired, dec.Reason)
    case approval.ActionPrompt:
        ok, err := r.approvalMgr.Prompter().PromptYesNo(ctx,
            fmt.Sprintf("Approve %s?", tool.Name()), false)
        if err != nil { return nil, err }
        if !ok {
            return nil, fmt.Errorf("%w: user declined at
        prompt", approval.ErrApprovalRequired)
        }
    case approval.ActionAllow:
        // proceed
    }
    if dec.SandboxRequired {
        if params == nil { params = map[string]interface{}{} }
        params["_helix_sandbox_required"] = true
        params["_helix_sandbox_network_allowed"] = false
    }
}
// Existing F02 hook follows; F14 dispatch reads the marker.
```

The `yesnoAdapter` (5-line wrapper around F19's `askuser.Prompter`) returns a 2-choice Question {yes, no} and maps the result to bool; reuses F19's tty + retry + ctx cancellation handling.

Failing tests FIRST (focus on the production-code-path byte-evidence — most assertions land in T07's integration test, since the cmd/cli main wiring is hard to unit-test in isolation):

```
//go:build integration

func TestApproval_Integration_SuggestMode_DeniesWriteFile_FileNotCreated(t
        *testing.T) {
    tmpfile := filepath.Join(t.TempDir(), "p2f21-suggest")
    reg, mgr := buildIntegrationRegistry(t, approval.ModeSuggest)
    _ = mgr
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{
        "path": tmpfile, "content": "x",
    })
    require.Error(t, err)
    require.ErrorIs(t, err, approval.ErrApprovalRequired)
    _, statErr := os.Stat(tmpfile)
    require.True(t, os.IsNotExist(statErr), "file should NOT
        exist; gate must short-circuit")
}

func TestApproval_Integration_AutoEditMode_AllowsWriteFile_FileCreated(t
        *testing.T) {
    tmpfile := filepath.Join(t.TempDir(), "p2f21-autoedit")
    reg, _ := buildIntegrationRegistry(t, approval.ModeAutoEdit)
    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{
        "path": tmpfile, "content": "x",
    })
    require.NoError(t, err)
    data, err := os.ReadFile(tmpfile)
    require.NoError(t, err)
    require.Equal(t, "x", string(data))
}

func TestApproval_Integration_RuntimeModeChange_NextCallSeesNewMode(t
        *testing.T) {
    tmpfile := filepath.Join(t.TempDir(), "p2f21-runtime")
    reg, mgr := buildIntegrationRegistry(t, approval.ModeSuggest)

    _, err := reg.Execute(context.Background(), "write_file",
        map[string]interface{}{
        "path": tmpfile, "content": "x",
    })
    require.ErrorIs(t, err, approval.ErrApprovalRequired)
```

```
    mgr.Set(approval.ModeAutoEdit)

    _, err = reg.Execute(context.Background(), "write_file",
        map[string]interface{}{
        "path": tmpfile, "content": "y",
    })
    require.NoError(t, err)
    data, _ := os.ReadFile(tmpfile)
    require.Equal(t, "y", string(data))
}

// Plus: PromptYes/No (auto-edit + run), F02 final-deny override
        (PHASE-E),
// FullAuto + bwrap (skip-with-marker if missing).
```

Subject: `feat(P2-F21-T07): main.go wiring (--approval pflag + manager + registry hook + /approval) + integration test.`

Task 8: Challenge harness (5 phases, positive byte evidence)

Files: new `helix_code/tests/integration/cmd/p2f21_challenge/main.go`, new `challenges/p2-f21-codex-approval-modes/CHALLENGE.md`, new `challenges/p2-f21-codex-approval-modes/run.sh`.

Harness phases (per spec §6.3):

1. **PHASE-A: SUGGEST-DENIES-EDIT (always runs)** — real `ApprovalManager(ModeSuggest)` + real registry + real `WriteFileTool` → call `registry.Execute("write_file", ...)` → assert (i) returned error wraps `ErrApprovalRequired`, (ii) `os.Stat(target)` returns `IsNotExist`, (iii) WriteFileTool's spy invocation count == 0 (gate short-circuited).
2. **PHASE-B: AUTO-EDIT-ALLOWS-EDIT-PROMPTS-RUN (always runs)** — real `ApprovalManager(ModeAutoEdit)` + yes-replying Prompter → write_file succeeds (file content == expected); shell tool with yes-prompter runs (output captured); re-wire with no-replying Prompter → shell tool returns `ErrApprovalRequired` AND exec wrapper invocation count == 0.
3. **PHASE-C: FULL-AUTO-FORCES-SANDBOX (skip-with-marker if no bwrap)** — assert `sandbox.Manager.Capabilities().HasSandbox == true`; real `ApprovalManager(ModeFullAuto)` + real `sandbox.Manager` → call `registry.Execute("shell", ...)`; assert (i) params marker `_helix_sandbox_required == true`, (ii) `_helix_sandbox_network_allowed == false`, (iii) sandbox.Manager.Execute spy invocation count == 1, (iv) command output captured, (v) network-attempt command exits non-zero.
4. **PHASE-D: RUNTIME-MODE-CHANGE (always runs)** — start with `ModeSuggest` → write_file → `ErrApprovalRequired` + file does not exist; call `manager.Set(ModeAutoEdit)`; write_file → no error + file exists with expected content; assert `manager.Mode() == ModeAutoEdit` after.
5. **PHASE-E: F02-DENY-OVERRIDES-F21-ALLOW (always runs)** — `ApprovalManager(ModeDangerous)` (F21 would allow everything) + real F02

PolicyEngine with rule denying writes to `/etc/` → call `registry.Execute("write_file", {path: "/etc/p2f21-E", content: "x"})` → assert (i) returned error wraps F02's denial reason, (ii) `/etc/p2f21-E` does NOT exist.

Output skeleton (verbatim per spec §6.3) ends with:

```
SUMMARY: PHASE-A=5/5 PASS; PHASE-B=6/6 PASS; PHASE-C=8/8 PASS [or SKIP-OK:
         PHASE-D=7/7 PASS; PHASE-E=5/5 PASS
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into `07_phase_2_evidence.md`. Dual commit (Challenges submodule + meta-repo bump).

`challenges/p2-f21-codex-approval-modes/run.sh` mirrors F19/F20 structure: `cd HelixCode && go run ./tests/integration/cmd/p2f21_challenge/main.go`.

Subject: `feat(P2-F21-T08): challenge with 5 phases (suggest deny + auto-edit prompt + full-auto sandbox + runtime change + F02 final-deny) — positive byte evidence.`

Task 9: Close-out + push

Tick all 9 items in PROGRESS, advance PROGRESS focus from F21 to "Phase 2 of CLI-Agent Fusion programme: F21 closed; F22 next candidate". Run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/approval internal/commands/approval_command.go &&
        echo BLUFF || echo clean
go test -count=1 ./internal/approval/...
go test -count=1 ./internal/commands/...
go test -count=1 ./internal/tools/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy
git diff --exit-code go.mod  # MUST be no-op (zero new deps)
git diff --exit-code go.sum  # MUST be no-op
```

Cross-compile check (matches F19/F20):

```
cd HelixCode && GOOS=linux GOARCH=amd64 go build -o /tmp/
        helixcode-linux-amd64 ./cmd/server
ls -la /tmp/helixcode-linux-amd64  # confirm produced binary
```

Commit `chore(P2-F21-T09): close out feature 21 — Codex Approval Modes`. Push 4 remotes non-force (`origin`, `helixdev`, `vasic-digital`, `gitlab` per programme conventions). Request explicit user authorization at this step (CONST-043).

PROGRESS.md milestone entry (verbatim):

```
- 2026-05-06 — Feature 21 (Codex Approval Modes) closed. 9 task commits (T
  Real, end-to-end 4-mode codex-compatible approval system: suggest (read-
  / auto-edit (edit OK, run prompts) / full-auto (edit + run with sandbox
  + network DENY) / dangerously-bypass (no checks; 2-second pause + warnin
  Selector: --approval flag > HELIXCODE_APPROVAL env > ~/.config/helixcode
  > default suggest. Per-tool Tool.RequiresApproval() ApprovalLevel (read-
  edit/run/all); ~30 existing tools migrated with explicit levels; safe-de
  LevelEdit for forgotten overrides. /approval slash (status/set/show); ru
  mode swap via atomic.Pointer takes effect on next CheckApproval. F02 ret
  final-deny authority (composition truth table). Zero new external deps.
  [5-phase Challenge evidence summary]. **First Phase 2 feature shipped.**
```

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + descriptors (§3.3 + §3.4 4×4 matrix), T03 selector (§3.6 precedence), T04 manager + YAML loader (§3.3 + §3.7), T05 Tool interface + ~30-tool migration (§3.6 per-tool table), T06 /approval slash (§4.5), T07 main.go wiring (§4.1 startup) + integration test (§6.2), T08 Challenge five phases (§5.2 + §6.3), T09 close-out (§9 + Phase 2 first-feature milestone).
2. **TDD:** every code task starts with failing tests. Types test pins §3.4's 4×4 matrix byte-for-byte (no recomputation — the constant array literal MUST equal the spec table). Selector tests use `func(string) string` closure (no `os.Setenv` ever — pure-function tests). Manager tests use a `fakeTool` value type + atomic.Pointer reads with `require.Equal` on Decision. Migration test (T05) enumerates every registered tool against an expected-level table. /approval command tests use a constructed manager value type (no mocks). Integration tests wire the production `Selector` + `ApprovalManager` end-to-end through the real registry, real F02, real F14 (skip-with-marker for bwrap).
3. **Type consistency:** `ApprovalMode`, `ApprovalLevel`, `Action`, `Decision`, `ResolvedSource`, `ManagerOptions`, `ApprovalManager`, `Tool`, `YesNoPrompter`, `DefaultLevelEdit`, `SelectorInput`, sentinel errors (`ErrApprovalRequired`, `ErrUnknownMode`, `ErrSandboxRequired`), constants (`ModeSuggest`/`ModeAutoEdit`/`ModeFullAuto`/`ModeDangerous`, `LevelReadOnly`/`LevelEdit`/`LevelRun`/`LevelAll`, `ActionAllow`/`ActionDeny`/`ActionPrompt`, `SourceFlag`/`SourceEnv`/`SourceConfig`/`SourceDefault`), command name `approval`, env var `HELIXCODE_APPROVAL`, flag `--approval`, YAML key `mode` — all match across spec §3.3 and plan T02–T07.
4. **Zero new external deps:** stdlib + existing testify/zap + `gopkg.in/yaml.v3` (already direct from F20). `go mod tidy` after T07 produces no diff in `go.mod` or `go.sum`. T09's verification step asserts `git diff --exit-code go.{mod,sum}` is no-op.
5. **Anti-bluff (§5.2):** Challenge has FIVE phases (PHASE-C uses skip-with-marker for bwrap-absent hosts; the marker is a registered SKIP-OK ticket, not silent-green). Every phase records positive evidence: invocation counters (PHASE-A: WriteFileTool counter == 0; PHASE-B: exec counter == 0 in no-reply scenario), filesystem-state assertions (PHASE-A: `os.IsNotExist`; PHASE-B/D: `os.ReadFile` content equality), Prompter-replay equality (PHASE-B yes/no branches), sandbox-spy invocation count (PHASE-C count == 1), runtime-mode-change observable in tool outcome (PHASE-D), F02-final-deny composition (PHASE-E). The four real-execution criteria — (a) suggest mode actually denies edits with no side-effect; (b) auto-edit prompts on run, denies on no-reply; (c) full-auto routes through F14 sandbox with network-deny marker; (d) runtime mode change reflected in next call — each have dedicated unit + integration + Challenge assertions. Coverage test (T05) ensures no tool registered without a level. Byte-evidence mismatch is a hard Challenge failure.

6. **CONST-042:** approval YAML carries no secrets (the chosen mode is not sensitive). The manager's logger NEVER logs the tool params at INFO level. A unit test scans `internal/approval/*.go` for `logger\.\bInfo\(.*\b(params|command|args|body)\b` matches and FAILs on any hit (defensive log discipline; consistent with F19/F20's stance even though there are no actual secrets).
7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T09 before pushing.
8. **CONST-033:** F21 emits no shell commands of its own. `full-auto` + `LevelRun` routes USER-supplied commands through F14, which has its own CONST-033 deny-list. F21 inherits that protection.
9. **Non-obvious call: separate `internal/approval/` package vs extending `internal/tools/permissions/`** (recorded in spec §11 #1). F02 is rule-based (path globs); F21 is risk-posture-based (single enum). Conflating bloats both layers; separate keeps each simple. Composition rule (§4.4) gives the user F02's rule precision AND F21's mode breadth.
10. **Non-obvious call: F21 gate runs BEFORE F02** (recorded in spec §11 #2). F21 is a coarse posture filter; F02 is fine-grained rules. Running F21 first short-circuits the "user is in suggest mode" case without asking F02 to enumerate every rule. F02 retains final-deny authority because its rules are persistent.
11. **Non-obvious call: tool default `LevelEdit` (safe-by-default)** (recorded in spec §11 #3 + plan T05 explicit note). A new tool that forgets to override gets gated, not bypassed. The migration test catches forgotten overrides for **existing** tools; the embedded `DefaultLevelEdit` catches **future** additions automatically.
12. **Non-obvious call: atomic-pointer mode swap** (recorded in spec §11 #4). Lock-free, single writer + many readers; the 4×4 matrix is constant data so no locks needed. Anti-bluff (d) is structurally impossible because every CheckApproval begins with `mode := *m.mode.Load()`.
13. **Non-obvious call: 2-second pause before `dangerously-bypass`** (recorded in spec §11 #5). Deliberate friction, injectable seam for tests, unconditional in production v1.
14. **Non-obvious call: `full-auto` forces network DENY** (recorded in spec §11 #6). Network-allowed full-auto would be a prompt-injection footgun. Users who need network use `dangerously-bypass` (which is honest about the risk).
15. **Non-obvious call: F02 final-deny composition** (recorded in spec §11 #7 + spec §4.4 truth table). F21's `Allow` does NOT override F02's `Deny`; the test in T07 (PHASE-E) asserts both directions of the composition.
16. **Non-obvious call: `Prompter` is F19's interface (wrapped via `YesNoPrompter`)** (recorded in spec §11 #8). Reused, not re-implemented; the wrapper is a 5-line adapter that maps yes/no to F19's 2-choice question. T07 reuses F19's tty + retry + ctx cancellation handling.
17. **Non-obvious call: `ModeDescriptors` is static data, not computed** (recorded in spec §11 #9). The 4×4 matrix is the source of truth; T02's test pins each row byte-for-byte against §3.4.
18. **Non-obvious call: sandbox-required marker in `params` map** (recorded in spec §11 #10). Avoids changing `Tool.Execute`'s signature; the marker is invisible to tools that don't care about sandboxing. T07's PHASE-C asserts the marker is read by F14's existing dispatch.
19. **Non-obvious call: `full-auto` aborts startup if sandbox unavailable** (recorded in spec §11 #11). Silent downgrade would be a security bluff. T07's wiring asserts the abort path with a fake SandboxMgr whose `Capabilities().HasSandbox=false`.
20. **Non-obvious call: `/approval set` does NOT write back to YAML** (recorded in spec §11 #12). Ephemeral semantics; misclick doesn't persist. F21.5 may add `--persist`.
21. **Non-obvious call: MCP tools default to `LevelEdit`** (recorded in spec §11 #13). Conservative; F21.5 will read MCP capability metadata for per-server level.

22. **First Phase 2 feature:** F21 is the inaugural Phase 2 feature. T01 advances PROGRESS.md from "Phase 1 complete" to "Phase 2 in flight" + creates `docs/improvements/07_phase_2_evidence.md` (the new evidence section for Phase 2). T09's milestone entry includes "**First Phase 2 feature shipped.**".
23. **T07 reuses F12 selector pattern, no new infrastructure:** the `--approval` pflag is added to the existing pflag set (one new line near other flags); the env-var read uses `os.Getenv` (or the existing env-getter wrapper if one exists); the YAML loader follows F12's `wizard_writer.go::LoadWizardConfig` shape. NO new selector framework or registry needed.